

Recursion

Eunbyung Park
Yonsei University



Textbook Chapter 11

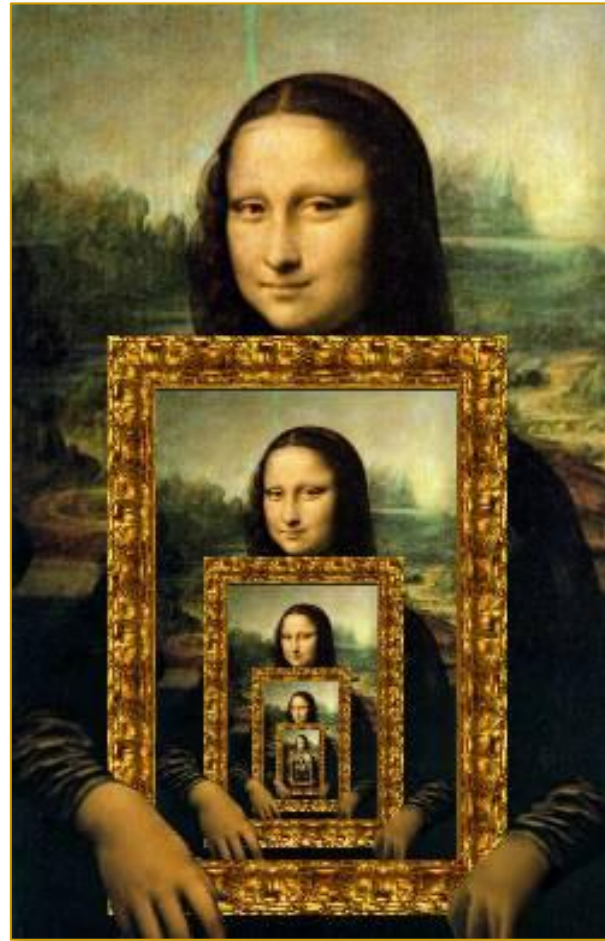
Recursive Functions

*Disclaimer: the following slides are provided by the course instructor.
Copyright 2026, Yonsei University.*

These course materials have been produced and published for the purpose of teaching Yonsei University students, so they cannot be used for any other purposes nor be shared with others. The legal responsibility for the violation lies with the actor him/herself.

“To iterate is human, to recurse divine.”

(L. Peter Deutsch)



Outline

- Recursion
- Divide and Conquer

Recursion (informally)

“To understand recursion, you need to understand recursion first.”

Definition of the acronym “GNU”:

GNU is not Unix

This is a recursive acronym 😊

From Merriam-Webster:

Recursion: a computer programming technique involving the use of a procedure, subroutine, function, or algorithm that calls ***itself*** one or more times until a specified condition is met at which time the rest of each repetition is processed from the last one called to the first — compare *iteration*.

What is Recursion?

- **Recursion** occurs when a procedure calls itself to solve a **simpler** version of the problem. With each recursive call, the problem is different from, and simpler than, the original problem.
- At some stage, the problem is so simple that we can solve it — the recursion **terminates**.
 - This is the **base-case** of the recursion
- Previous problem instance solved in terms of the simple sub-problem(s).
- And so on, back to the initial procedure call...

Recursion Example – Solving Factorials

- The problem of solving factorials is our first example of recursion.
- The factorial operation (!) in mathematics is illustrated below.

$$1! = 1$$

$$2! = 2 * 1$$

$$3! = 3 * 2 * 1$$

$$4! = 4 * 3 * 2 * 1$$

or $2 * 1!$

or $3 * 2!$

or $4 * 3!$

$$1! = 1$$

Base Case

$$n! = n * (n-1)!$$

Recursive Case

where n is non-negative

Notice that each successive line can be solved in terms of the previous line. For example, 4! is equivalent to the problem

$$4 * 3!$$

Using A Recursive Function

- A recursive function to solve the factorial problem is given below. Notice the recursive call in the last line.
- The function **calls itself** to solve a **smaller version** of the factorial problem.

```
def fact(n):  
    """  
    Return the value of n!  
    precondition: n >= 1  
    """  
    if n == 1:  
        return 1  
    else:  
        return n * fact(n - 1)
```

- The **base case** is a fundamental situation for which we know the answer.
 - In the case of finding factorials, the answer of 1! is by definition 1.
 - No further work is needed for the base case.
- The **recursive case** is computed from the solution of a smaller version of the problem, plus some additional computations.

Recursion – Step By Step

Suppose we call the method to solve fact(4).

- This will result in **four** calls of method fact:

fact(4): This is not the base case ($n \neq 1$), so we return the result of $4 * \text{fact}(3)$. This multiplication will not be carried out until an answer is found for fact(3). This leads to the second call to solve fact(3).

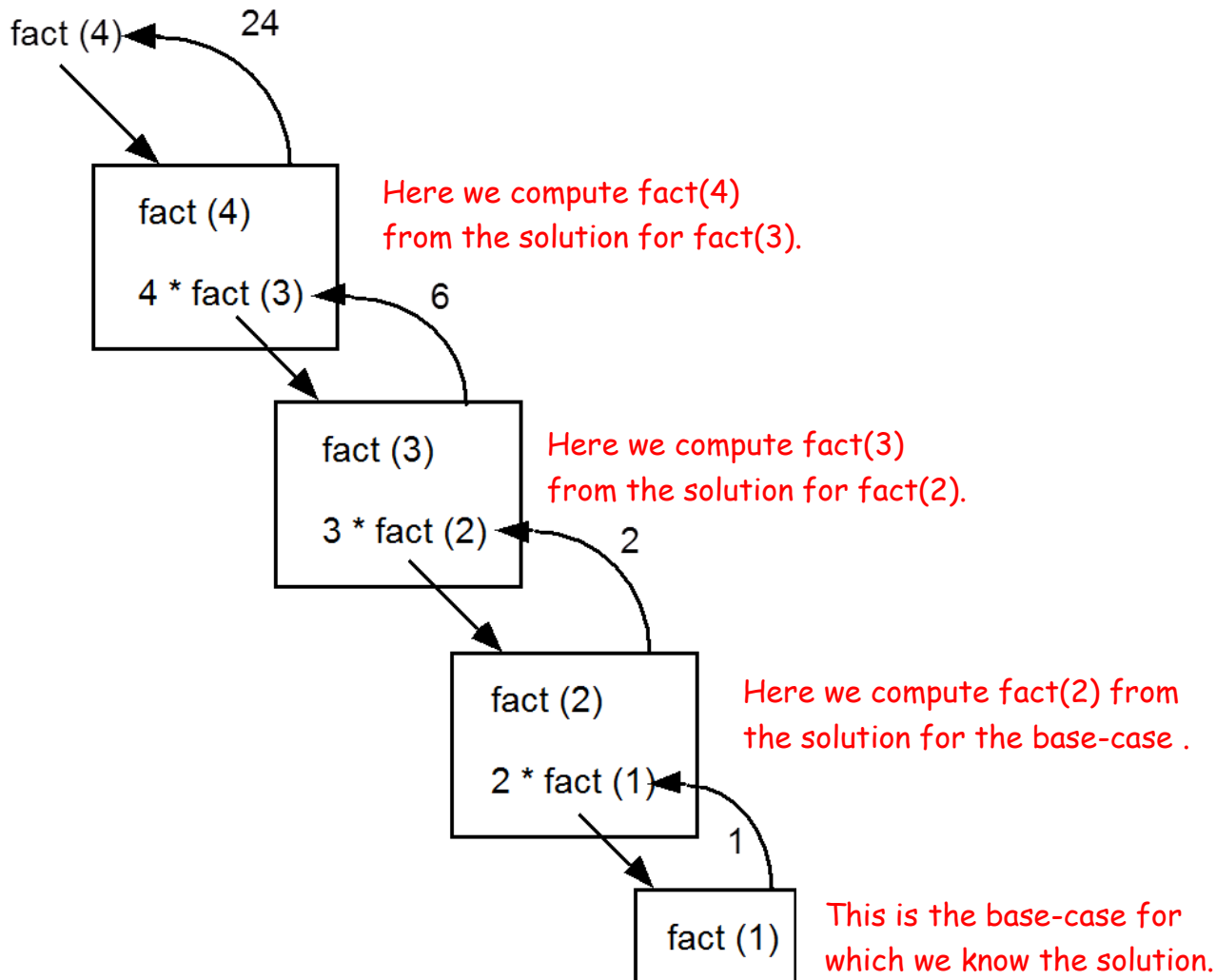
fact(3): Again, this is not the base case and we return $3 * \text{fact}(2)$. This leads to another recursive call to solve fact(2).

fact(2): Still, this is not the base case, we must solve $2 * \text{fact}(1)$.

fact(1): Finally, we reach the base case, which returns the value 1.

```
def fact(n):  
    if n == 1:  
        return 1  
    else:  
        return n * fact(n - 1)
```

Recursion – The Big Picture



- Each box represents a call of function `fact()`. To solve `fact(4)` requires four calls to function `fact()`.
- Notice that when the recursive calls were made inside the **else** statement, the value fed to the recursive call was $(n-1)$. This is where the problem is getting **smaller** and **simpler** with the eventual goal of solving `1!`.

Recursion – Step By Step

- When a recursive call is made, the current computation is temporarily suspended with all its current information available for later use.
 - Same as for any other function call.
- A completely new copy of the function's data is used to evaluate the recursive call (Python: a new frame is created),
 - using a fresh set of local variables,
 - and its own set of function arguments.
- When the recursive call is completed, the value returned by the recursive call is used to complete the suspended computation. The suspended computation's work now proceeds.
- When the base case is encountered, the recursion will **terminate** by returning a value to the caller. The caller will then return a value to the caller's caller aso until the final answer becomes available in the initial recursive call.

Infinite Recursion

- If the recursion **never reaches the base case**, the recursive calls will continue until the computer runs out of memory. Python raises the exception **RecursionError** in such a case.



fact(-1)

```
def fact(n):  
    if n == 1:  
        return 1  
    else:  
        return n * fact(n - 1)
```

- When programming recursively, you need to make sure that the algorithm is **moving towards the base case**. Each successive call of the algorithm must be solving a simpler version of the problem.
- A recursive computation that does not proceed towards its base case will attempt to run forever.
 - Called **infinite recursion**.
 - Results in a run-time error.

Recursion versus Iteration

Recursion and iteration are equally ``powerful":

- Any recursive algorithm can be implemented iteratively (with a loop), but sometimes only with great difficulty.
 - Recursion works well to design certain kind of algorithms (**divide and conquer algorithms**)
 - See examples on following slides.
- However, a recursive solution will (almost) always run more slowly than an iterative one because of the overhead of function calls.
 - Tail recursion and related compiler optimizations can mitigate this overhead.
- Functional programming languages favor recursion over iteration
 - Don't support loops, or only as an ``afterthought".
- OOP and procedural programming languages support iteration and recursion.

Divide and Conquer

- Break the problem into several subproblems that are similar to the original problem but smaller in size, solve subproblems recursively, and combine solutions to create solution to the original problem.

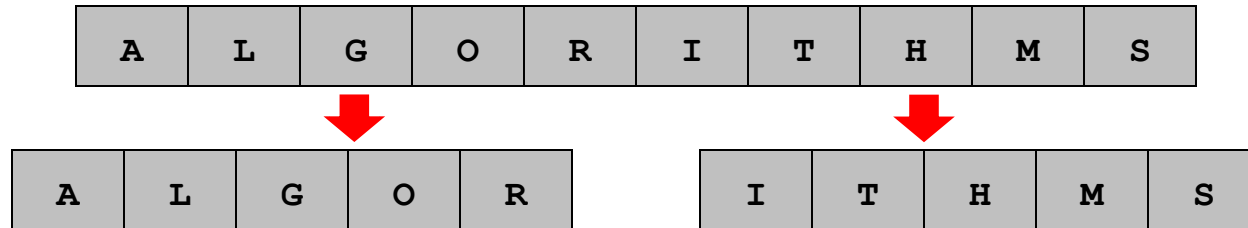
At each level of the recursion:

- 1) **Divide** the problem into a number of subproblems.
- 2) **Conquer** subproblems:
 - directly if subproblem is simple enough (base case)
 - recursively otherwise
- 3) **Combine** subproblem solutions into solution for the original problem.

Mergesort

1) **Divide** array into 2 halves:

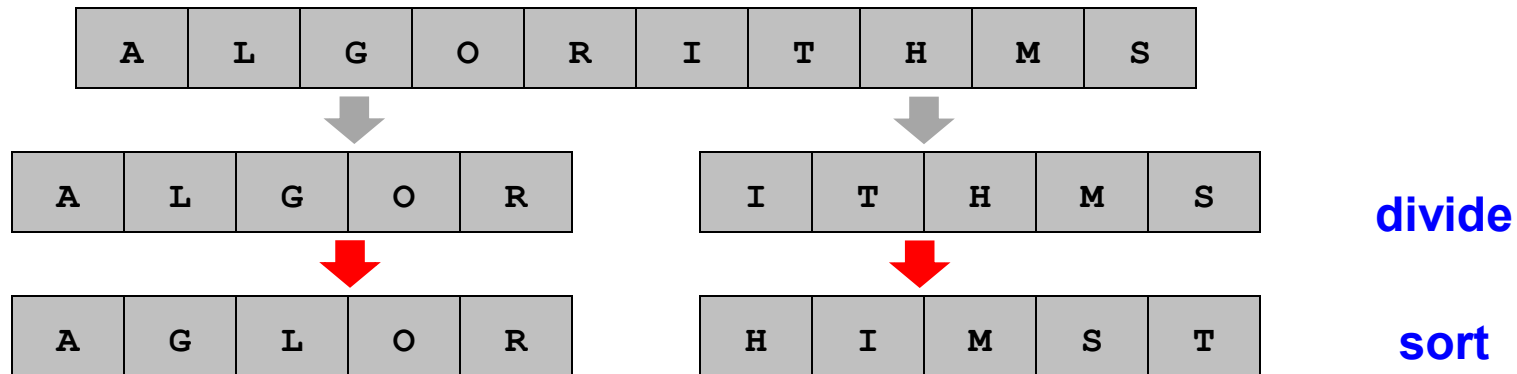
Assume we want to sort an array of characters, integers or floats.



divide

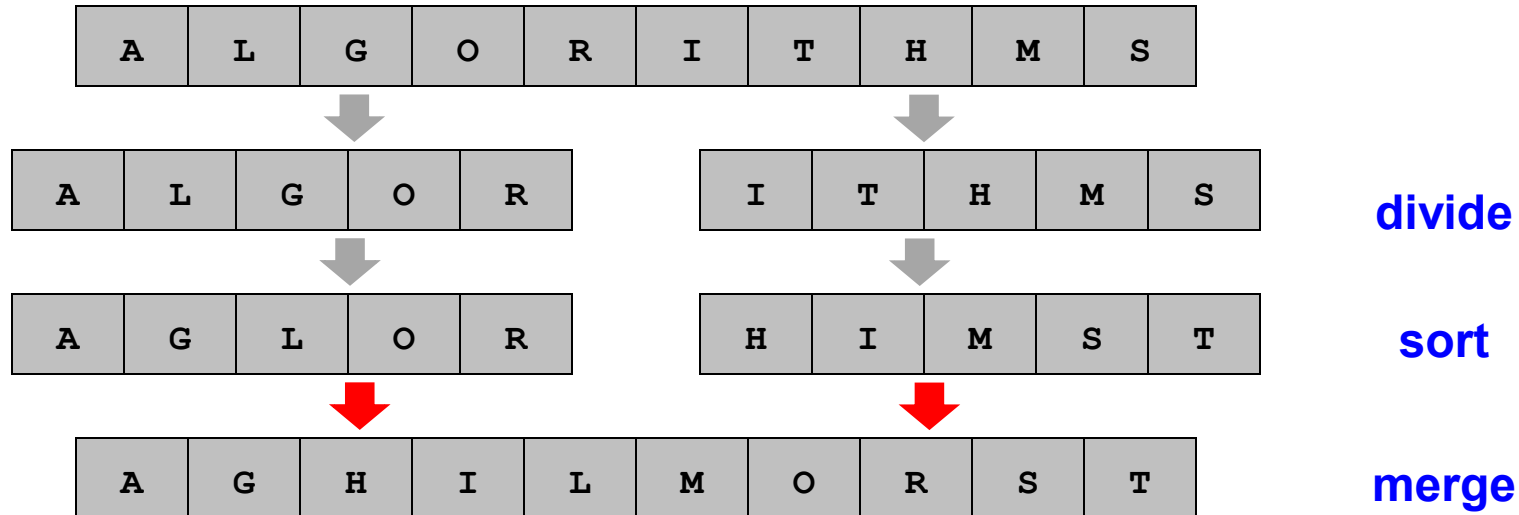
Mergesort

- 1) **Divide** array into 2 halves.
- 2) **Conquer**: recursively sort each half:



Mergesort

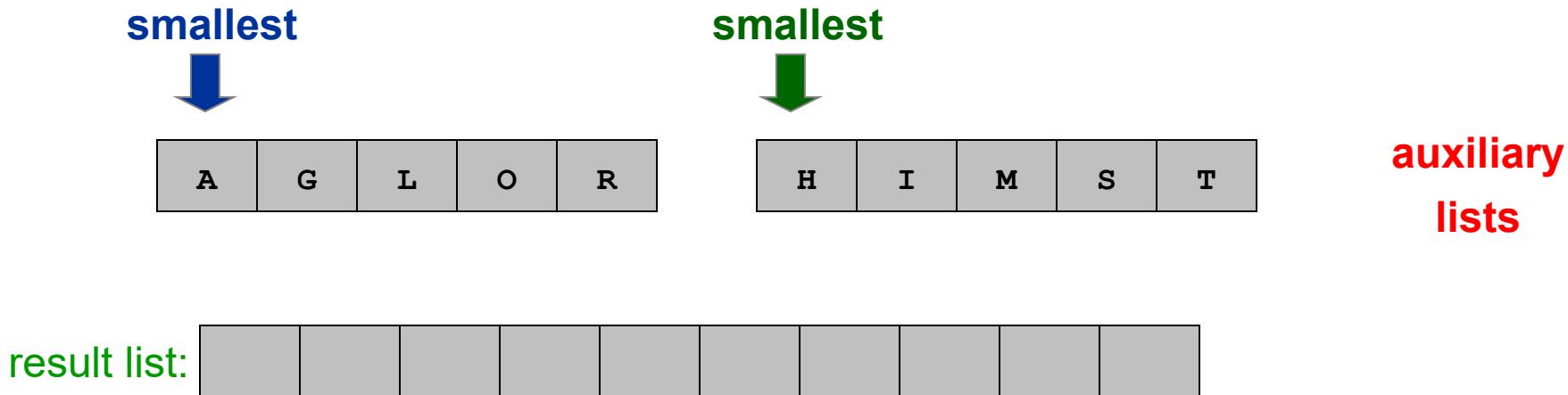
- 1) **Divide** array into 2 halves.
- 2) **Conquer**: recursively sort each half.
- 3) **Combine**: merge two halves to make sorted whole:



Mergesort Combine Step

Combine:

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.



Mergesort Combine Step

Combine:

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.

smallest



A	G	L	O	R
---	---	---	---	---

smallest



H	I	M	S	T
---	---	---	---	---

auxiliary
lists

result list:

A									
---	--	--	--	--	--	--	--	--	--

Mergesort Combine Step

Combine:

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.

smallest



A	G	L	O	R
---	---	---	---	---

smallest



H	I	M	S	T
---	---	---	---	---

result list:

A	G								
---	---	--	--	--	--	--	--	--	--

Mergesort Combine Step

Combine:

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.

smallest



A	G	L	O	R
---	---	---	---	---

smallest



H	I	M	S	T
---	---	---	---	---

result list:

A	G	H							
---	---	---	--	--	--	--	--	--	--

Mergesort Combine Step

Combine:

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.

smallest



A	G	L	O	R
---	---	---	---	---

smallest



H	I	M	S	T
---	---	---	---	---

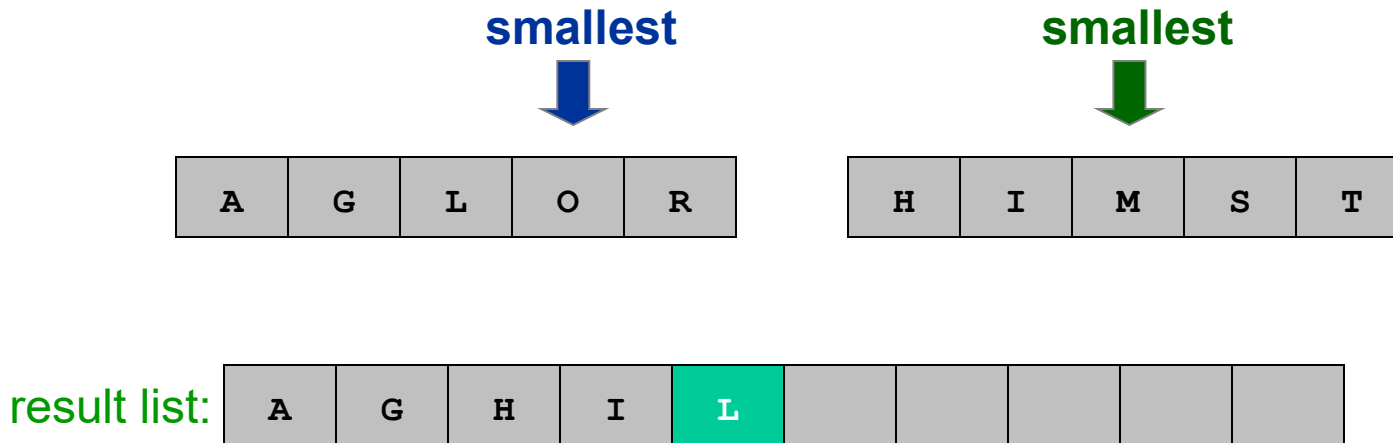
result list:

A	G	H	I						
---	---	---	---	--	--	--	--	--	--

Mergesort Combine Step

Combine:

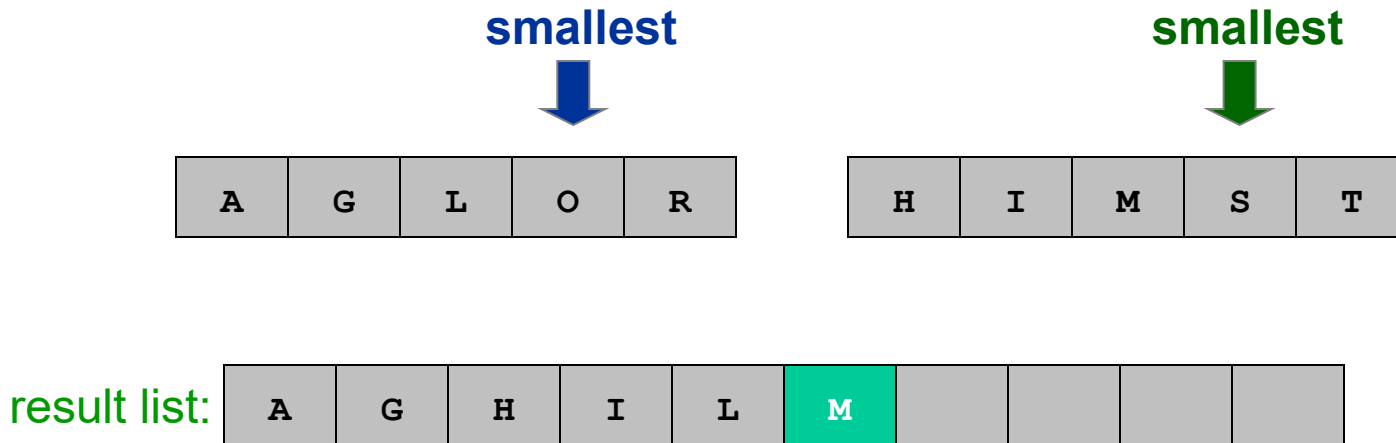
- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.



Mergesort Combine Step

Combine:

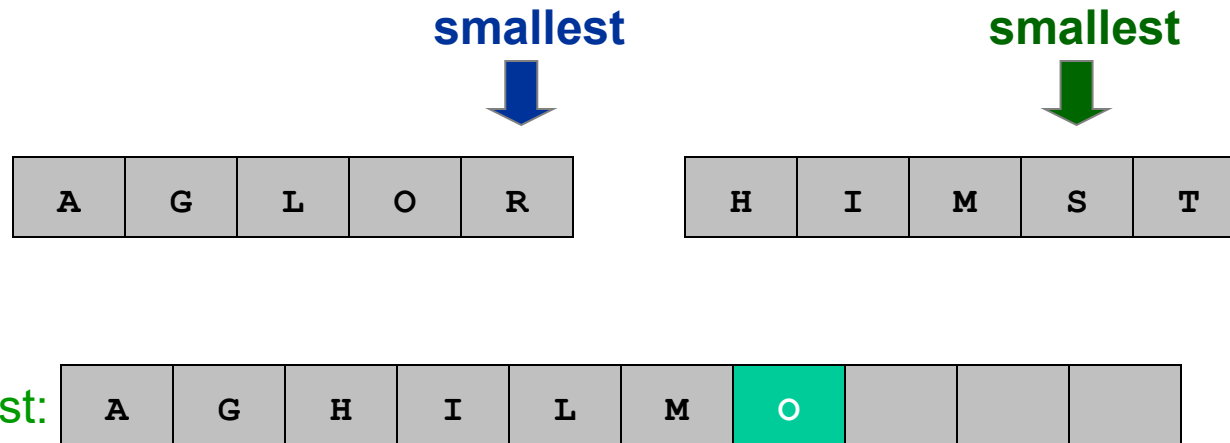
- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.



Mergesort Combine Step

Combine:

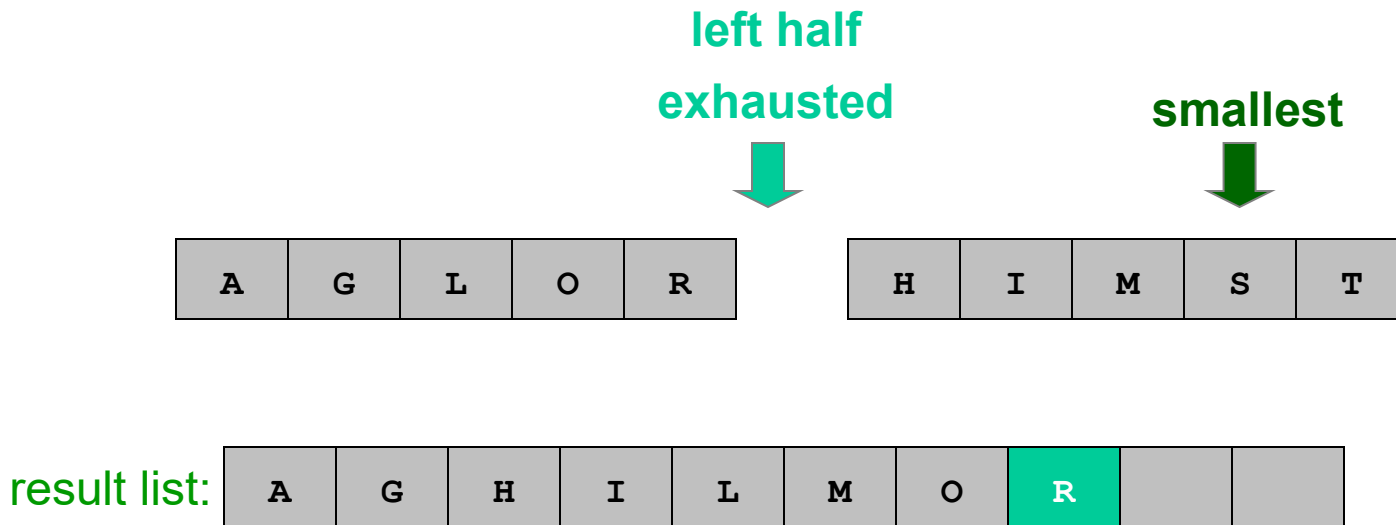
- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.



Mergesort Combine Step

Combine:

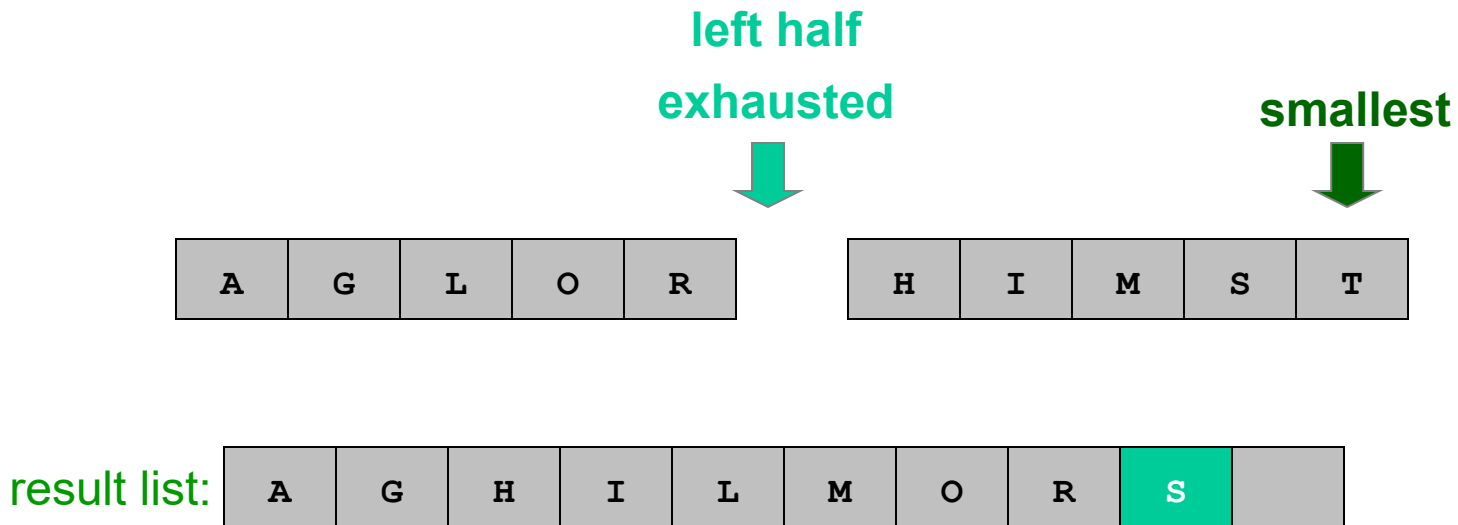
- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.



Mergesort Combine Step

Combine:

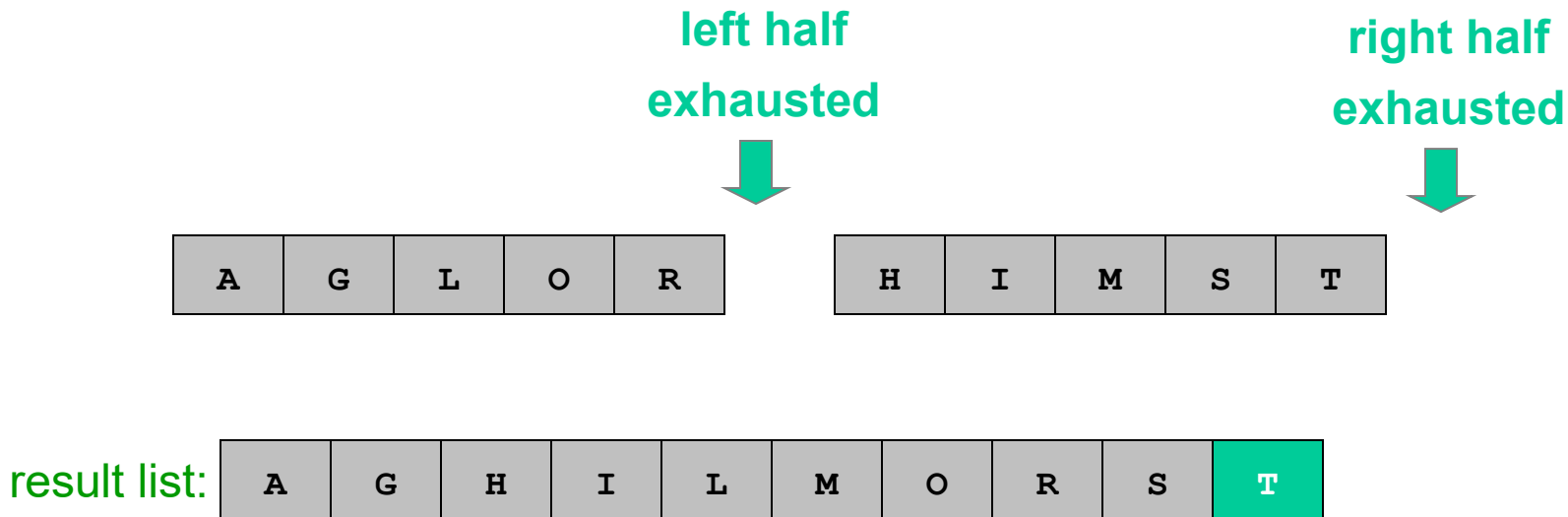
- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.



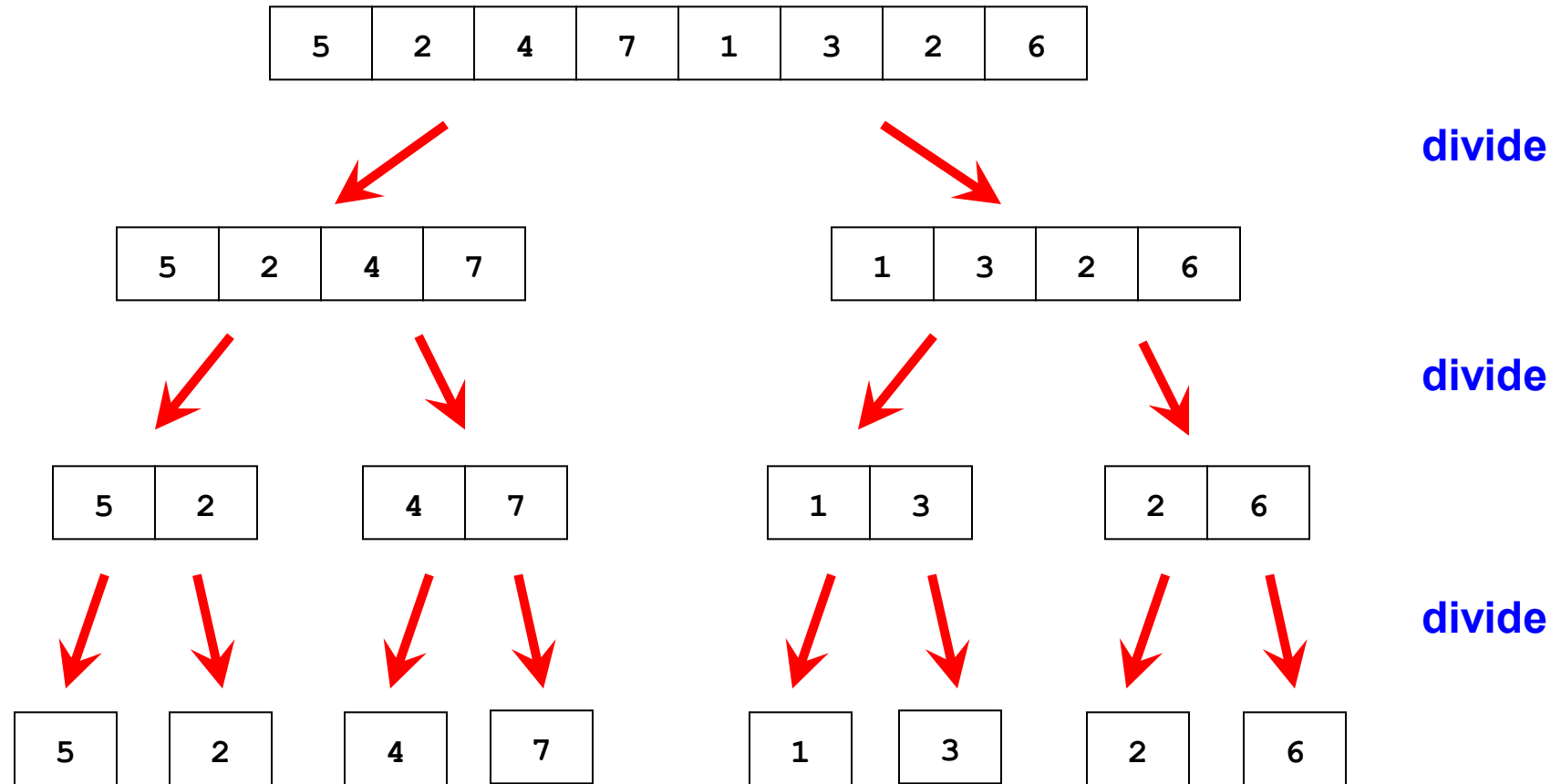
Mergesort Combine Step

Combine:

- Keep track of smallest element in each sorted half.
- Insert smallest of two elements into **result list**.
- Repeat until done.



Mergesort (Divide)



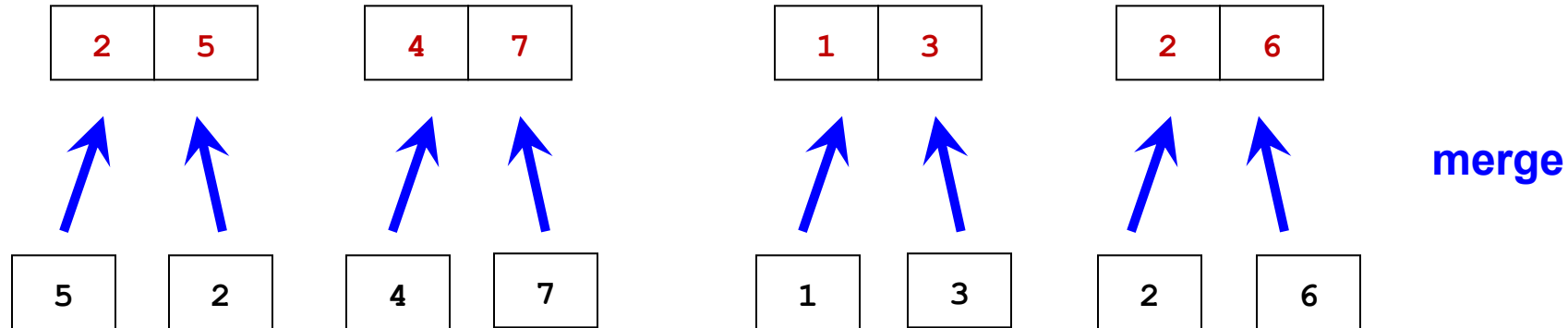
- Divide until we reach 1-element base-case.
 - One-element lists are **trivially sorted**.

Mergesort (Combine)

5	2	4	7	1	3	2	6
---	---	---	---	---	---	---	---

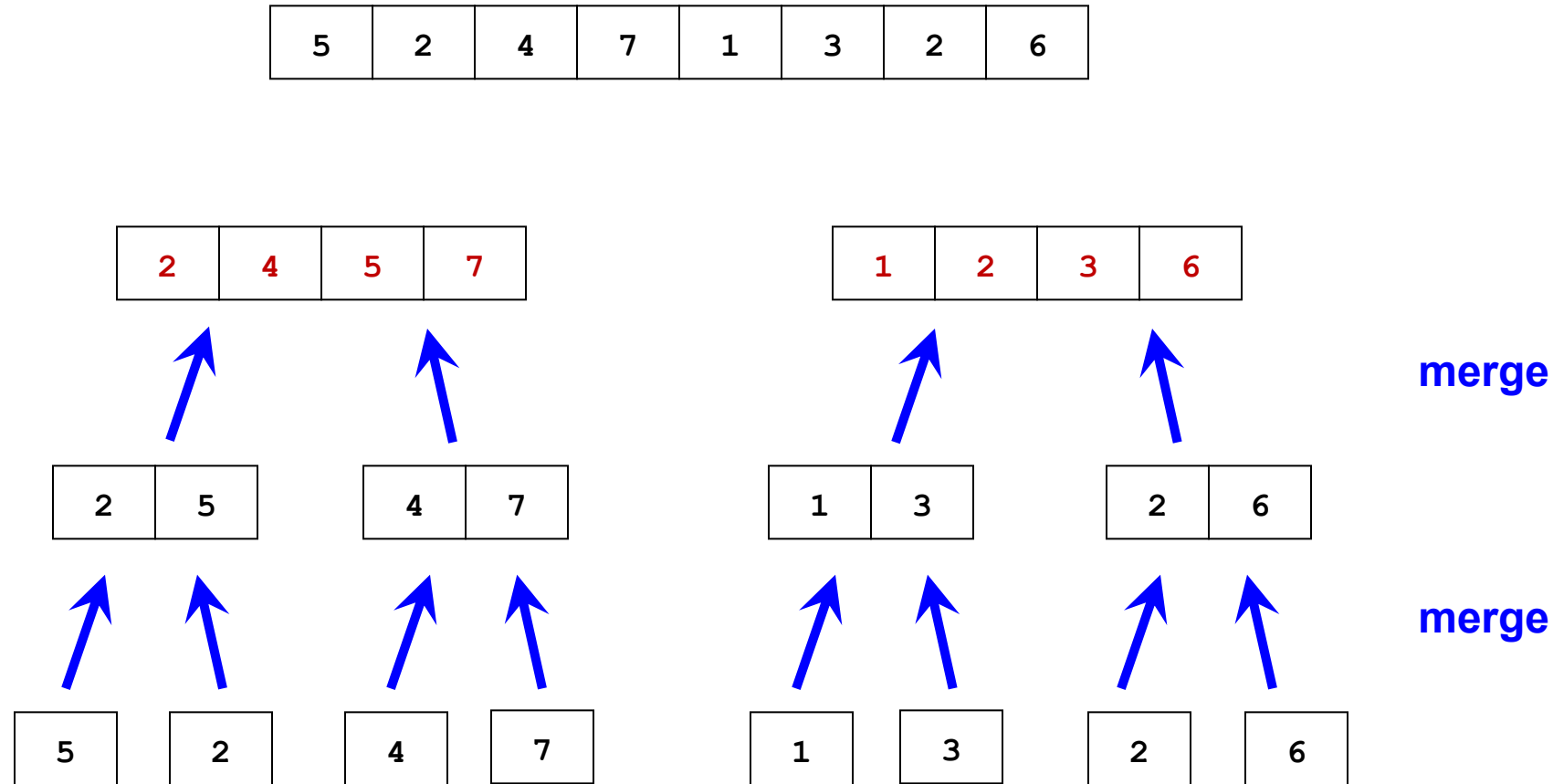
5	2	4	7
---	---	---	---

1	3	2	6
---	---	---	---



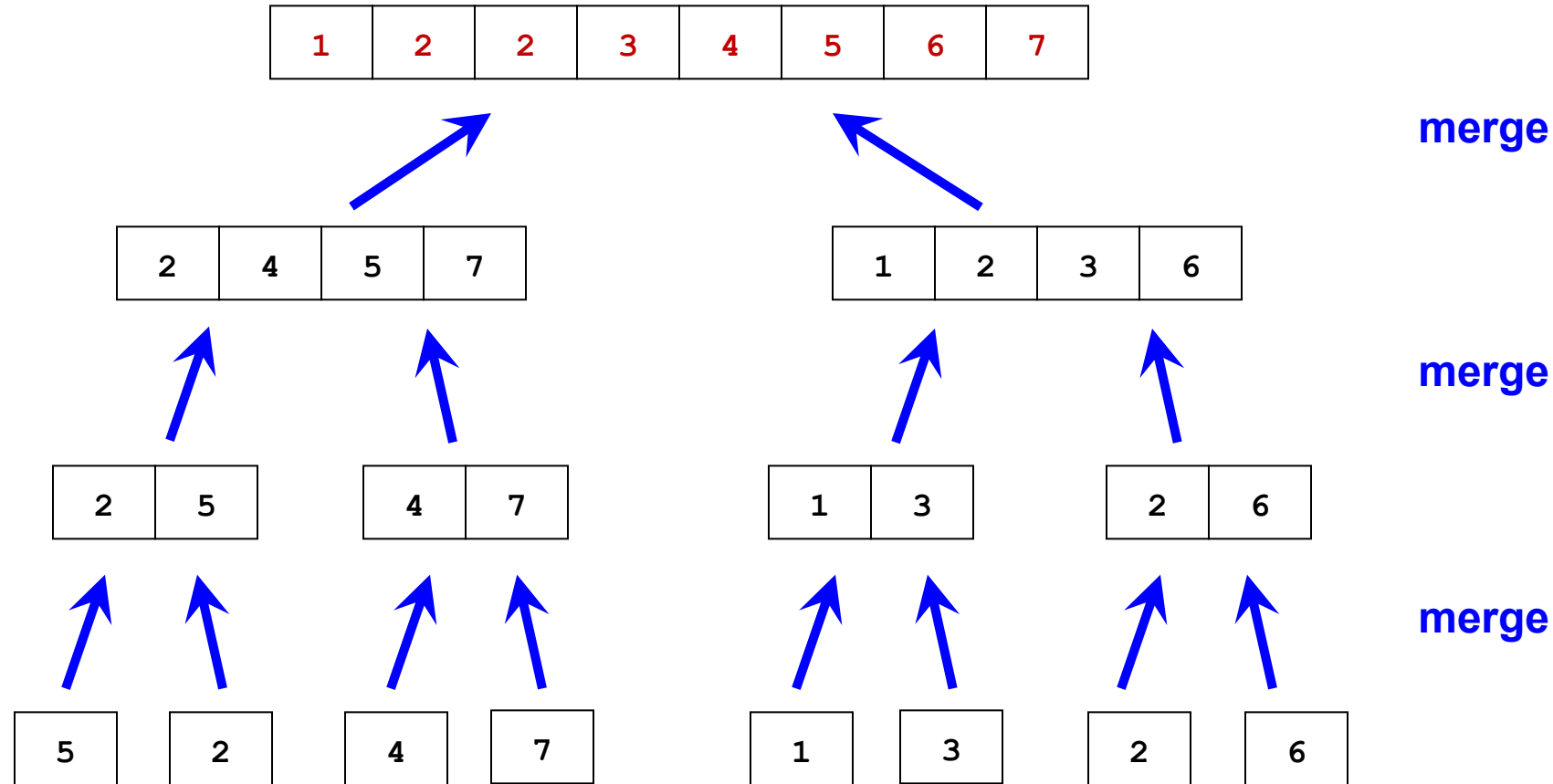
- Merge, starting from the 1-element base-cases,

Mergesort (Combine)



- ... merging the 2-element lists, ...

Mergesort (Combine)



- ... until we merge the final result list.


```

def mergeSort(slist):
    if len(slist) <= 1: # Trivially sorted, early exit:
        return
    mid = len(slist) // 2
    lefthalf = slist[:mid]
    righthalf = slist[mid:]

    mergeSort(lefthalf)
    mergeSort(righthalf)

    # Merge sorted halves:
    LeftCount = 0
    RightCount = 0
    GlobalCount = 0

    while LeftCount < len(lefthalf) and RightCount < len(righthalf):
        if lefthalf[LeftCount] < righthalf[RightCount]:
            slist[GlobalCount] = lefthalf[LeftCount]
            LeftCount += 1
            GlobalCount += 1
        else:
            slist[GlobalCount] = righthalf[RightCount]
            RightCount += 1
            GlobalCount += 1

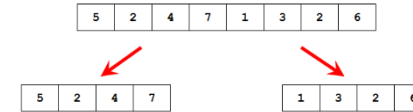
    # Collect remainder from left half (if any):
    while LeftCount < len(lefthalf):
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1

    # Collect remainder from right half (if any):
    while RightCount < len(righthalf):
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1

```

Implementing Mergesort

```
def mergeSort(slist):  
    if len(slist) <= 1: # Trivially sorted  
        return  
    mid = len(slist) // 2  
    lefthalf = slist[:mid]  
    righthalf = slist[mid:]  
  
    mergeSort(lefthalf)  
    mergeSort(righthalf) ...
```



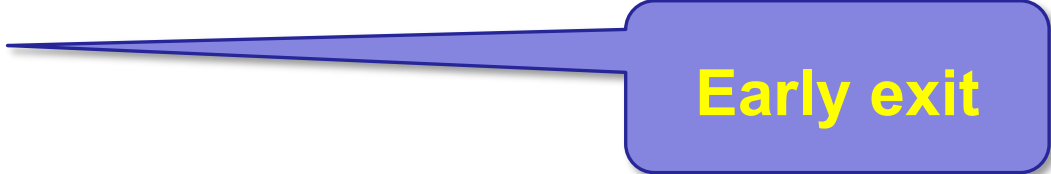
5	2	4	7	1	3	2	6	slist
0	1	2	3	4	5	6	7	index

mergeSort (slist)

mergeSort (slist[:4]) **mergeSort (slist[4:])**

Implementing Mergesort

```
def mergeSort(slist):  
    if len(slist) <= 1: # Trivially sorted  
        return  
    mid = len(slist) // 2  
    lefthalf = slist[:mid]  
    righthalf = slist[mid:]  
  
    mergeSort(lefthalf)  
    mergeSort(righthalf) ...
```



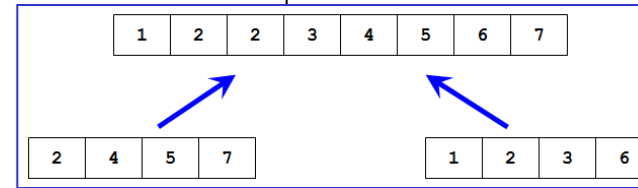
Early exit

- mergeSort is a recursive procedure
- If length of the to-be-sorted list **slist** larger than 1:
 - compute index **mid** of the middle element
 - **recursively** sort halves **slist[:mid]** and **slist[mid:]** and then **merge** sorted halves.
- If to-be-sorted list has length 1:
 - do nothing (base case, lists of length 1 are **trivially sorted**)

Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0
```

```
while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

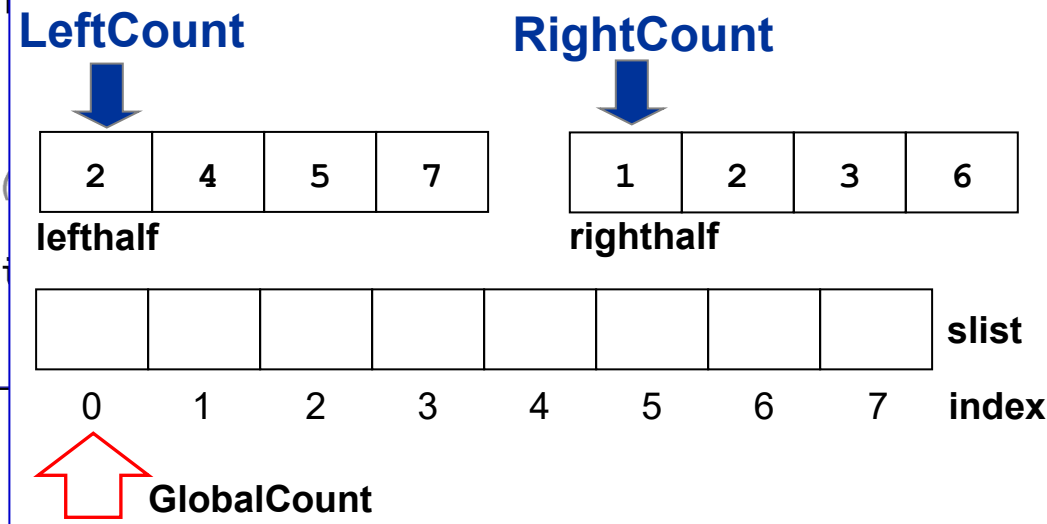


Collect remainder from left half (if any):

```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

Collect remainder from right half (if any):

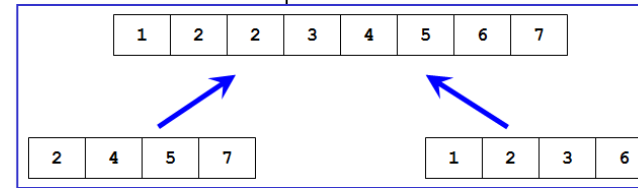
```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0
```

```
while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

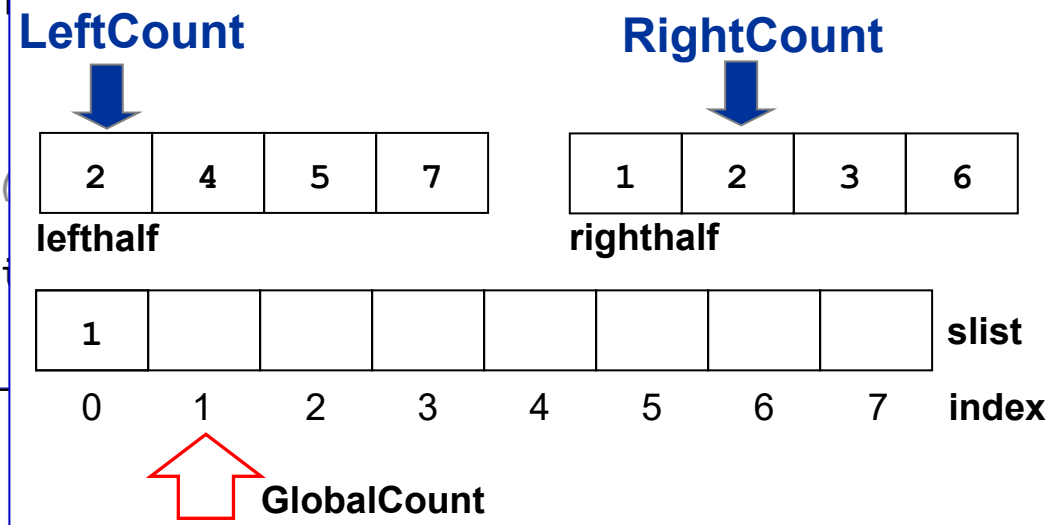


```
# Collect remainder from left half (if any):
```

```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

```
# Collect remainder from right half (if any):
```

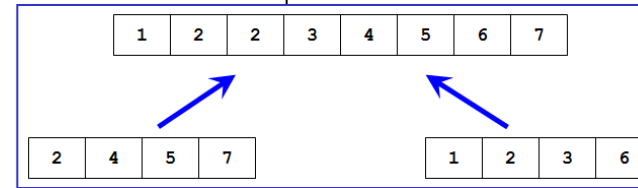
```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0
```

```
while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

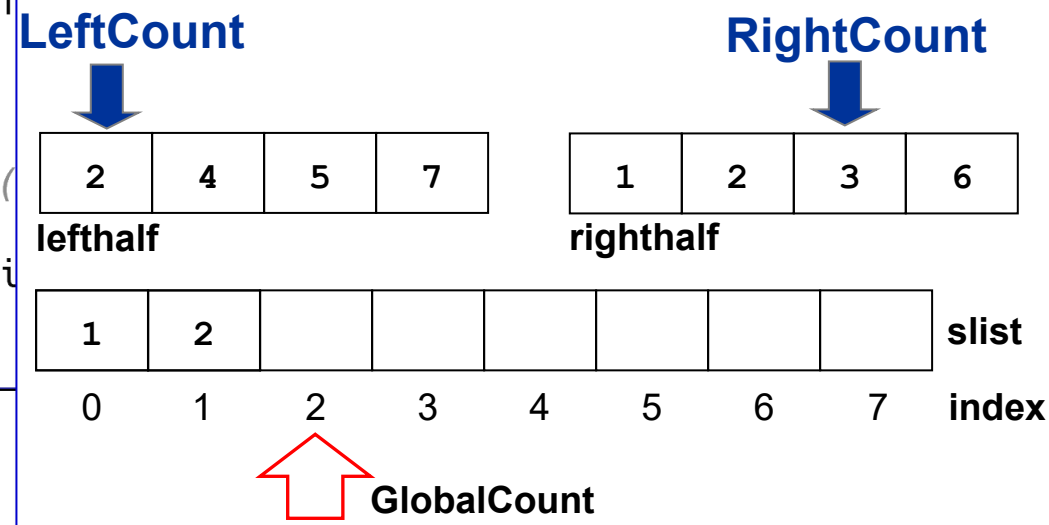


```
# Collect remainder from left half (if any):
```

```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

```
# Collect remainder from right half (if any):
```

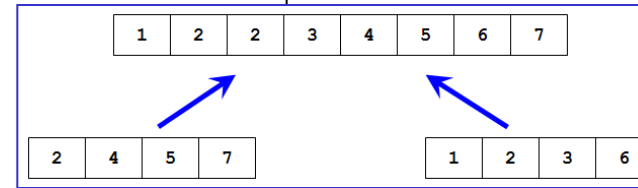
```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0

while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

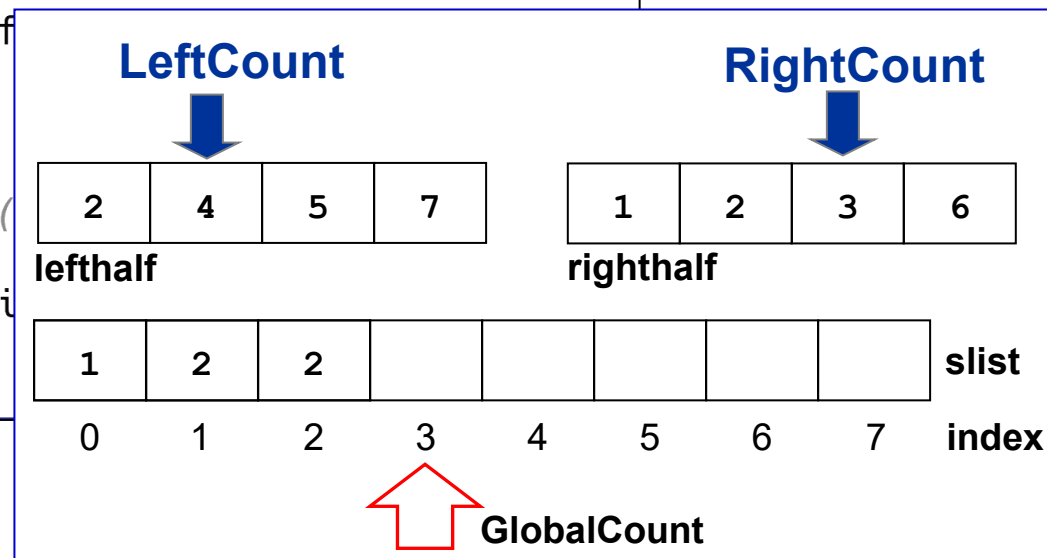


Collect remainder from left half (if any):

```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

Collect remainder from right half (if any):

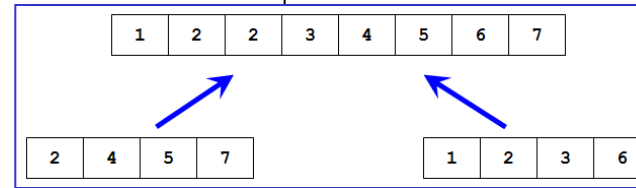
```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0

while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

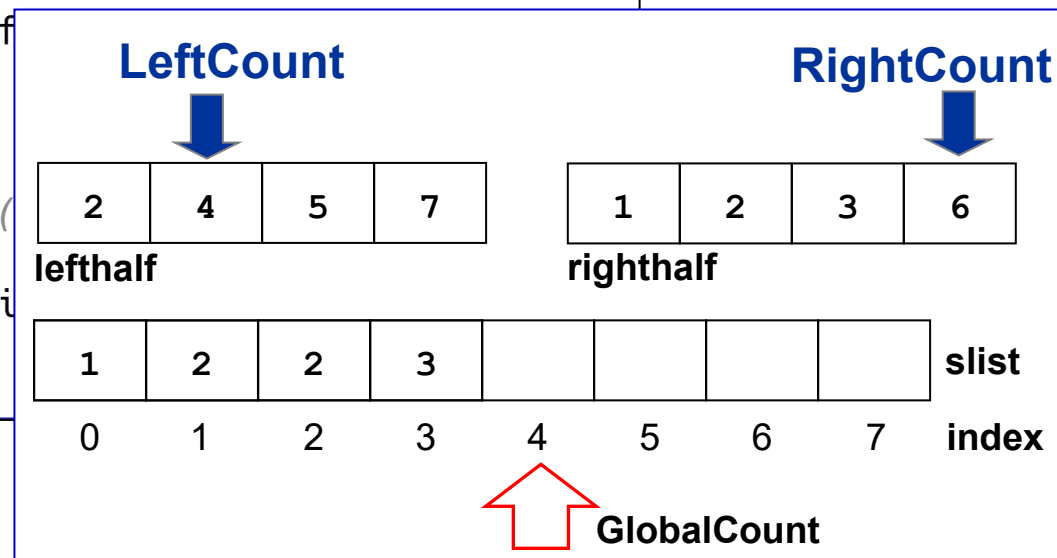


Collect remainder from left half (if any):

```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

Collect remainder from right half (if any):

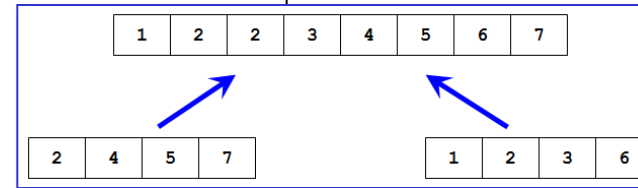
```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0

while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

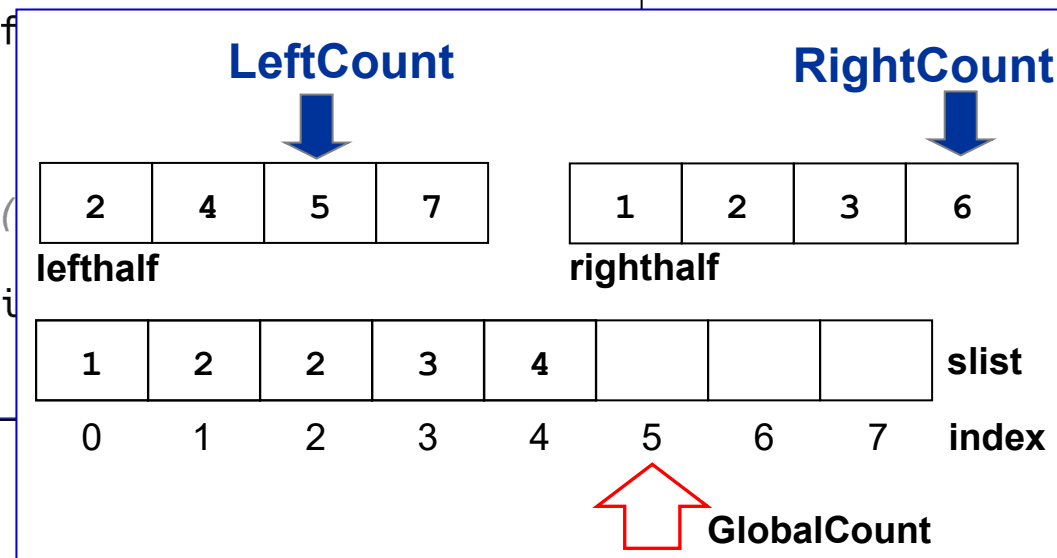


Collect remainder from left half (if any):

```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

Collect remainder from right half (if any):

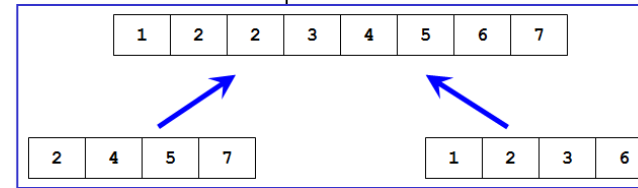
```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0

while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

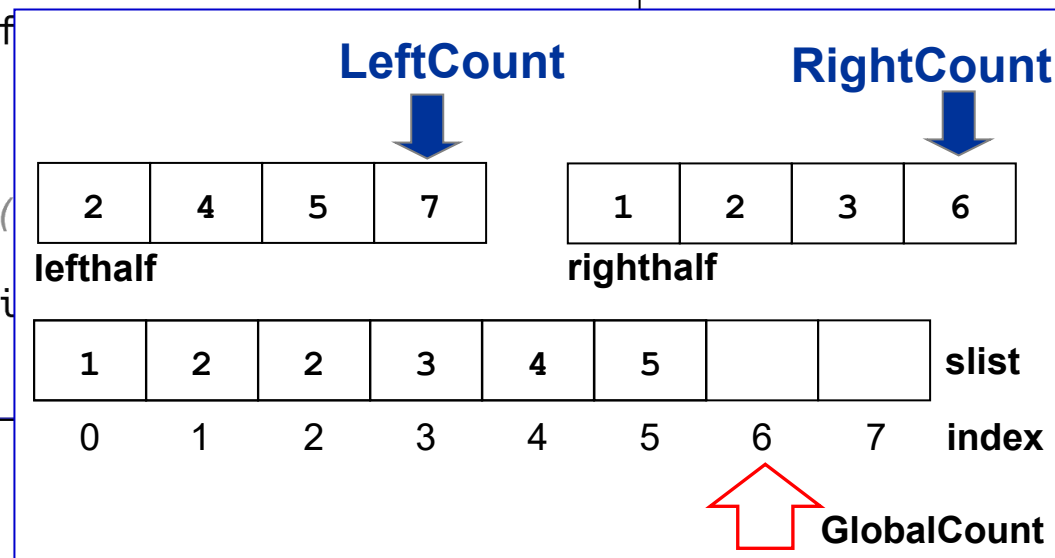


Collect remainder from left half (if any):

```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

Collect remainder from right half (if any):

```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0
```

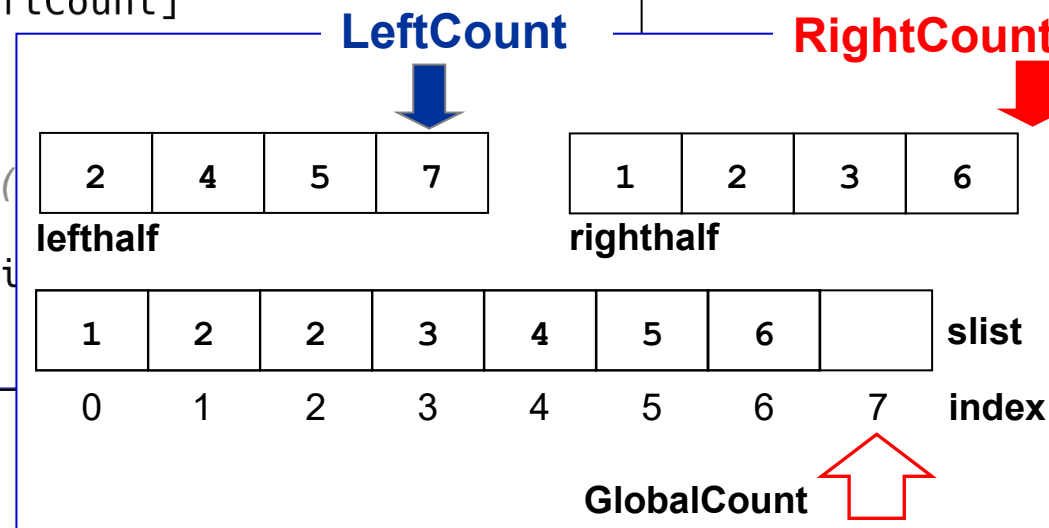
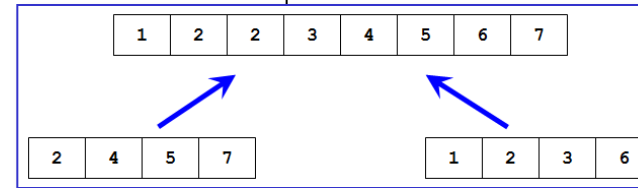
```
while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

Collect remainder from left half (if any):

```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

Collect remainder from right half (if any):

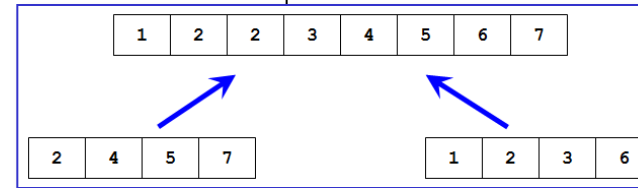
```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Implementing the Merge Step of Mergesort

```
LeftCount = 0
RightCount = 0
GlobalCount = 0

while LeftCount < len(lefthalf) and RightCount < len(righthalf):
    if lefthalf[LeftCount] < righthalf[RightCount]:
        slist[GlobalCount] = lefthalf[LeftCount]
        LeftCount += 1
        GlobalCount += 1
    else:
        slist[GlobalCount] = righthalf[RightCount]
        RightCount += 1
        GlobalCount += 1
```

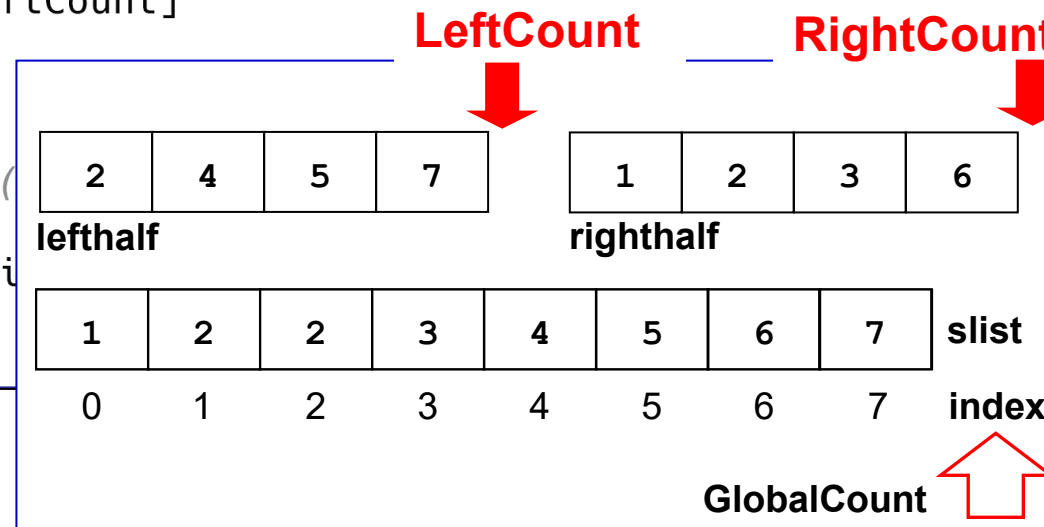


Collect remainder from left half (if any):

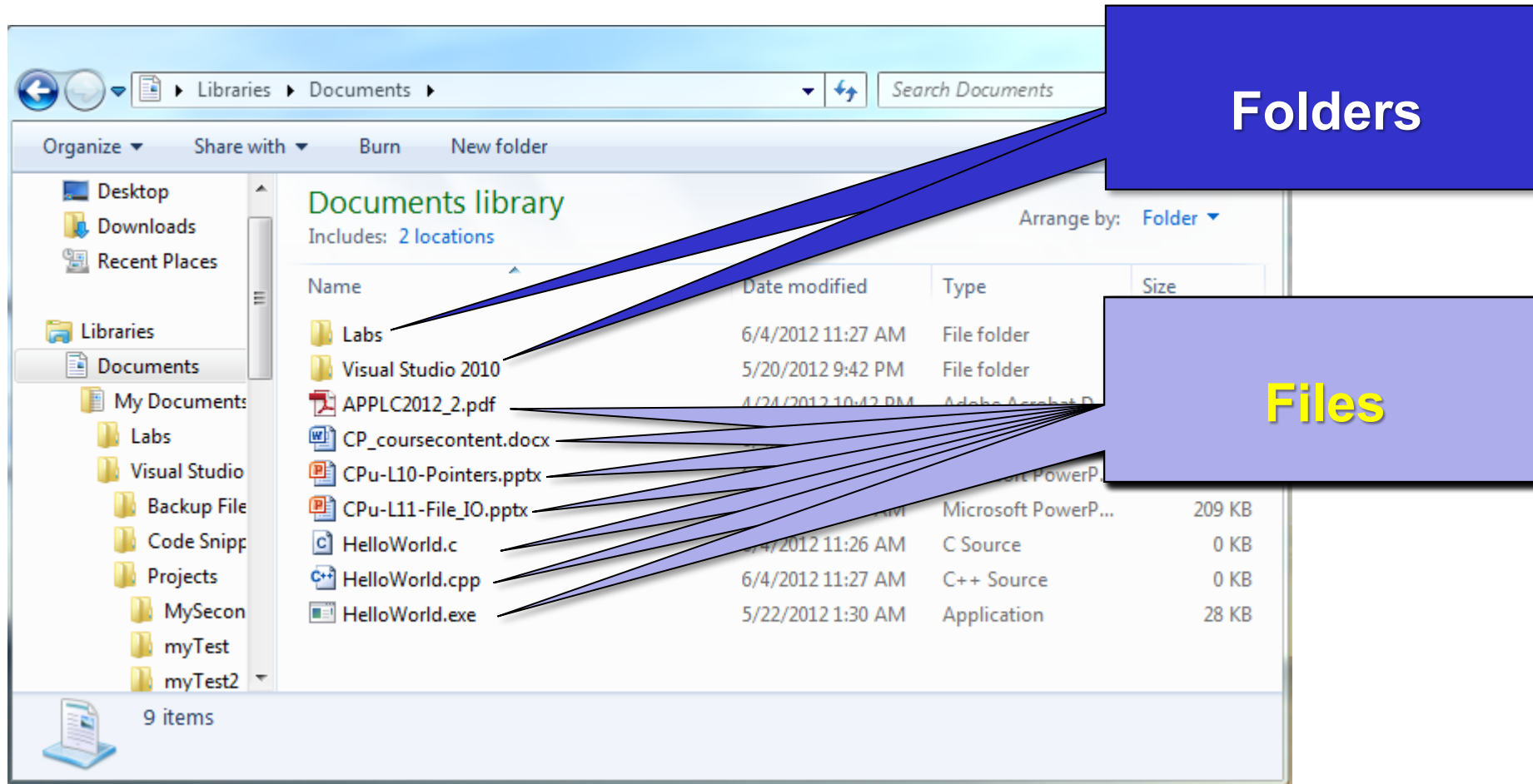
```
while LeftCount < len(lefthalf):
    slist[GlobalCount] = lefthalf[LeftCount]
    LeftCount += 1
    GlobalCount += 1
```

Collect remainder from right half (if any):

```
while RightCount < len(righthalf):
    slist[GlobalCount] = righthalf[RightCount]
    RightCount += 1
    GlobalCount += 1
```



Listing all Files Recursively



- Disks contain **folders**, a **folder** is a place to store files and other folders
 - This is a recursive definition ☺
- How do we process all files in a folder, including all the subfolders?
 - Use recursion!

Listing all Files Recursively

```
import os

def printAllFiles(directory):
    files = os.listdir(directory)
    for file in files:
        fullname = directory + '/' + file
        if os.path.isdir(fullname):
            printAllFiles(fullname) # recurse into subfolder 😊
        else:
            if os.path.isfile(fullname):
                print('File:', fullname)

printAllFiles('C:/Users/')
```

- The OS module allows us to use functionality provided by the operating system.
- Function-names are self-explanatory:
 - `listdir()` provides a list containing the names of files and folders in folder ``directory``.
 - `isdir()` returns True if a given name denotes a folder.
 - `isfile()` returns True if a given name denotes a file.

Listing all Files Recursively

```
def printAllFiles2(directory):  
    files = os.listdir(directory)  
    for file in files:  
        fullname = directory + '/' + file  
        try:  
            if os.path.isdir(fullname):  
                printAllFiles2(fullname)  
        else:  
            print('File:', fullname)  
    except OSError:  
        print('Access denied to:' + fullname)
```

- The above function is more robust against **access violations**.
- If our program tries to access a folder for which it does not have permissions, exception **OSError** will be raised.
- We catch this exception and give up on this folder.

Slide 830...



■ Here we are, having completed

- 1) Introduction
- 2) Data and Expressions
- 3) Control Structures
- 4) Lists
- 5) Functions
- 6) Object Use
- 7) Modular Design
- 8) Text files and Exceptions
- 9) Dictionaries and Sets
- 10) Object-oriented Programming, and
- 11) Recursion

I want to thank the TAs:

Junhyuk Park

Younghyun Kim

Hyunkyu Ko

Jungwoo Kim

Hwasik Jung

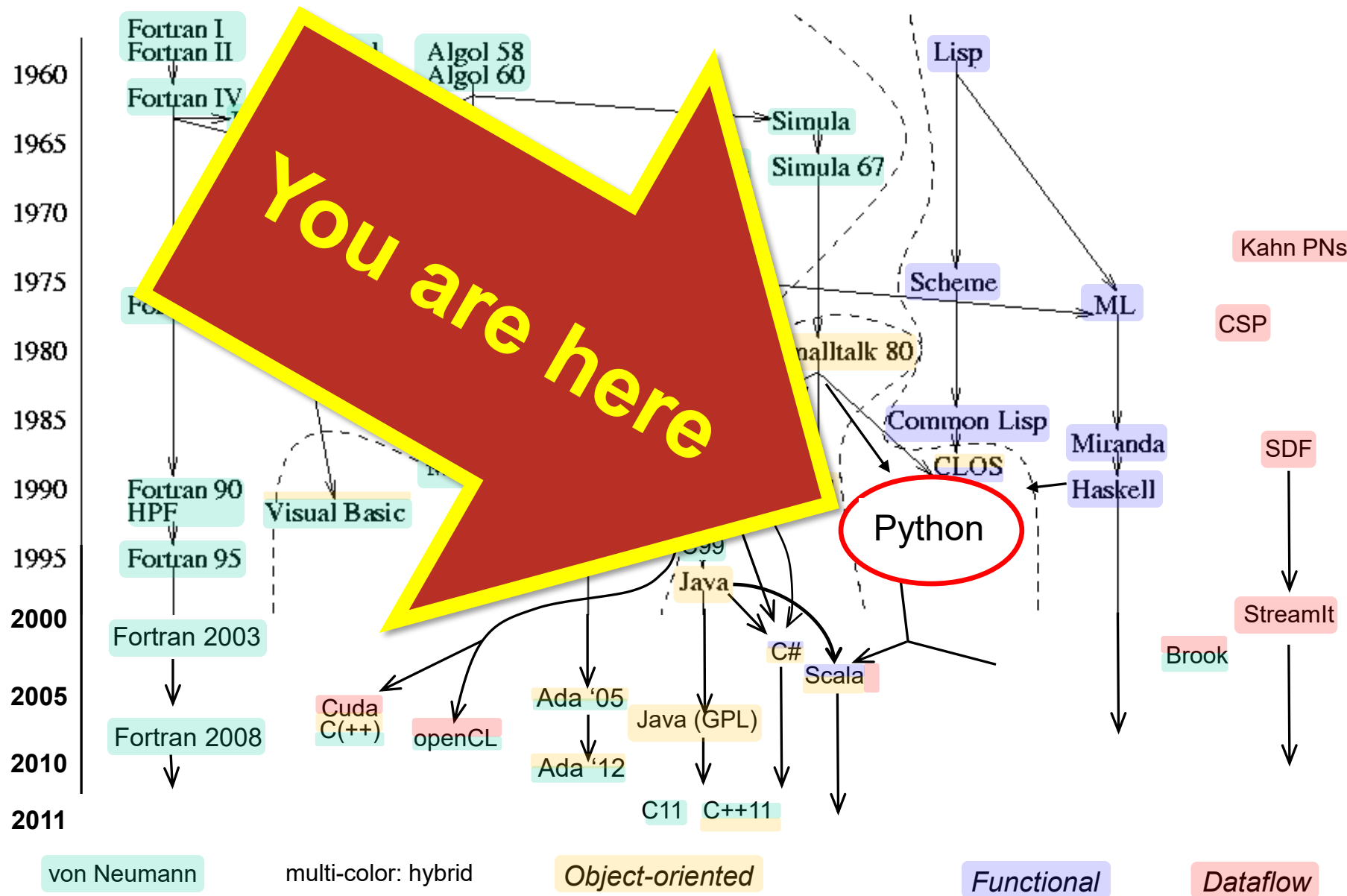
Byungjin kang

- This course is only possible because of the strong and uncompromising support from our TAs!
- Among many other tasks, they graded more than 4000 lab problems for us!

I want to thank you!

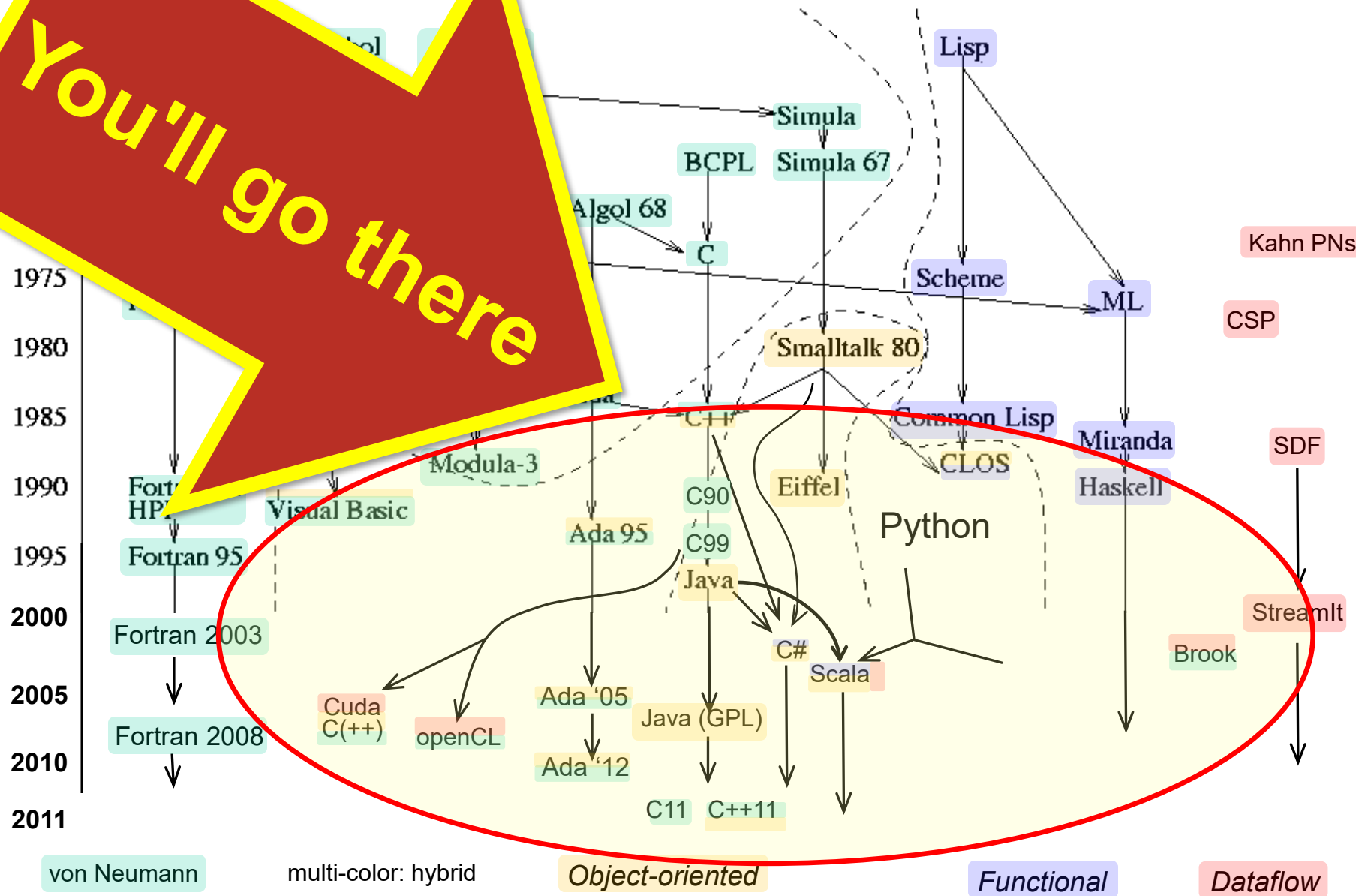
- You did a good job participating in this course!
- Now on to the final exam... 😊

Language Genealogy & Classification



Language Genealogy & Classification

You'll go there



After the course is
before the
course...

...but your second programming language will come to you already much easier than the first one...

You just keep coding...



Thank you very much!